

Practical Session 9

Task 1

Implement a file copy utility using low-level Linux file I/O system calls (open, read, write, lseek, close). Compare its performance with standard library functions (fopen, fread, fwrite).

```
j_rakshitha@RAKSHITHA:~/ossp_project$ cd ~
j_rakshitha@RAKSHITHA:~$ nano practical9_task1_copy.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra practical9_task1_copy.c -o practical9_task1_copy
j_rakshitha@RAKSHITHA:~$ ./practical9_task1_copy
=====
Practical Session 9 - Task 1
File Copy Utility
=====

Creating source file...

Low-level copy completed.
Standard-library copy completed.

Performance comparison:
Low-level I/O time      : 0.000063 seconds
Standard-library I/O time: 0.000029 seconds

Files created:
  practical9_lowlevel.txt
  practical9_standard.txt

Demonstrating lseek()...
File size using lseek(): 101 bytes

Task completed successfully.
j_rakshitha@RAKSHITHA:~$ cmp practical9_lowlevel.txt practical9_standard.txt
j_rakshitha@RAKSHITHA:~$
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 practical9_task1_copy.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/stat.h>
#include <time.h>

#define BUFFER_SIZE 4096

void low_level_copy(const char *source, const char *destination)
{
    int src_fd = open(source, O_RDONLY);

    if (src_fd == -1)
    {
        perror("open source");
        exit(1);
    }

    int dst_fd = open(destination, O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (dst_fd == -1)
    {
        perror("open destination");
        close(src_fd);
        exit(1);
    }

    char buffer[BUFFER_SIZE];
    ssize_t bytes_read;

    while ((bytes_read = read(src_fd, buffer, sizeof(buffer))) > 0)
    {
        ssize_t total_written = 0;

        while (total_written < bytes_read)
        {
            ssize_t bytes_written =
                write(dst_fd, buffer + total_written,
                    bytes_read - total_written);

            if (bytes_written == -1)
            {
                perror("write");
                close(src_fd);
                close(dst_fd);
                exit(1);
            }

            total_written += bytes_written;
        }
    }
}

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute  ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify  ^_ Go To Line M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 practical9_task1_copy.c
}

if (bytes_read == -1)
{
    perror("read");
}

close(src_fd);
close(dst_fd);
}

void standard_library_copy(const char *source, const char *destination)
{
    FILE *src = fopen(source, "rb");

    if (src == NULL)
    {
        perror("fopen source");
        exit(1);
    }

    FILE *dst = fopen(destination, "wb");

    if (dst == NULL)
    {
        perror("fopen destination");
        fclose(src);
        exit(1);
    }

    char buffer[BUFFER_SIZE];
    size_t bytes_read;

    while ((bytes_read = fread(buffer, 1, sizeof(buffer), src)) > 0)
    {
        if (fwrite(buffer, 1, bytes_read, dst) != bytes_read)
        {
            perror("fwrite");
            fclose(src);
            fclose(dst);
            exit(1);
        }
    }

    fclose(src);
    fclose(dst);
}

int main(void)
{
    const char *source = "practical9_source.txt";

    ^G Help    ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
    ^X Exit    ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 practical9_task1_copy.c

printf("=====\n");
printf("Practical Session 9 - Task 1\n");
printf("File Copy Utility\n");
printf("=====\n");

printf("\nCreating source file...\n");

FILE *file = fopen(source, "w");

if (file == NULL)
{
    perror("fopen");
    return 1;
}

fprintf(file,
        "Operating Systems and System Programming\n"
        "Practical Session 9\n"
        "Low-level Linux file I/O demonstration.\n");

fclose(file);

clock_t start = clock();

low_level_copy(source, "practical9_lowlevel.txt");

clock_t end = clock();

double low_level_time =
    (double)(end - start) / CLOCKS_PER_SEC;

start = clock();

standard_library_copy(source, "practical9_standard.txt");

end = clock();

double standard_time =
    (double)(end - start) / CLOCKS_PER_SEC;

printf("\nLow-level copy completed.\n");
printf("Standard-library copy completed.\n");

printf("\nPerformance comparison:\n");
printf("Low-level I/O time      : %.6f seconds\n",
        low_level_time);
printf("Standard-library I/O time: %.6f seconds\n",
        standard_time);

| printf("\nFiles created:\n");

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line M-E Redo
```

```

printf(" practical9_lowlevel.txt\n");
printf(" practical9_standard.txt\n");

printf("\nDemonstrating lseek()...\n");

int fd = open("practical9_lowlevel.txt", O_RDONLY);

if (fd == -1)
{
    perror("open");
    return 1;
}

off_t position = lseek(fd, 0, SEEK_END);

if (position == -1)
{
    perror("lseek");
    close(fd);
    return 1;
}

printf("File size using lseek(): %ld bytes\n",
      (long)position);

close(fd);

printf("\nTask completed successfully.\n");

return 0;
}

```

Task 2

Develop a program that redirects standard input and output using file descriptors and `dup2()`. Demonstrate how shells implement I/O redirection.

```

j_rakshitha@RAKSHITHA:~$ nano practical9_task2_dup2.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra practical9_task2_dup2.c -o practical9_task2_dup2
j_rakshitha@RAKSHITHA:~$ ./practical9_task2_dup2
Practical Session 9 - Task 2
dup2() I/O Redirection Demonstration

j_rakshitha@RAKSHITHA:~$ cat practical9_output.txt
This message is redirected to practical9_output.txt
Standard output (stdout) has been redirected using dup2().
j_rakshitha@RAKSHITHA:~$ |

```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 practical9_task2_dup2.c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main(void)
{
    int fd;

    printf("Practical Session 9 - Task 2\n");
    printf("dup2() I/O Redirection Demonstration\n\n");

    /* Redirect standard output to a file */
    fd = open("practical9_output.txt",
              O_WRONLY | O_CREAT | O_TRUNC, 0644);

    if (fd == -1)
    {
        perror("open");
        return 1;
    }

    if (dup2(fd, STDOUT_FILENO) == -1)
    {
        perror("dup2");
        close(fd);
        return 1;
    }

    close(fd);

    printf("This message is redirected to practical9_output.txt\n");
    printf("Standard output (stdout) has been redirected using dup2().\n");

    fflush(stdout);

    return 0;
}

[ Read 38 lines ]
^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```